COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Module 08: Calculus, mechanics and numerical simulation

Expanded Volume 02 / individual teaching module

Four lessons with derivations, runnable examples, practical work and 20 quiz questions.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Module 08 — Calculus, mechanics and numerical simulation

Original prerequisites: Modules 06.1 and 07.1; work through the linked calculus problem sets.

Original module project: Build a projectile and spring simulator with analytic comparison cases, timestep refinement, unit checks and a report separating model error from numerical error.

Expanded project: Projectile and spring laboratory. Analytic equations, tested particle/spring functions, timestamped trajectory CSVs, and a fixed-horizon refinement report comparing explicit and semi-implicit Euler.

Acceptance: Include h=0.1,0.05,0.025 at T=2 seconds for the m=1,k=100 undamped spring, plus a bounded unstable h=0.25 case. State maximum and final position errors, energy behavior, units, and omitted physics.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 08.1 — Limits, derivatives and rates

Learning outcome: Connect derivatives to velocity, acceleration and local approximation.

A derivative is a limit of average rates

For a position function x(t), the average velocity over a nonzero interval h is (x(t+h)-x(t))/h. The instantaneous velocity is the limit of that quotient as h approaches zero, when the limit exists. It describes the local behavior of a mathematical function, not an instruction to divide two identical floating-point samples by zero.

For x(t)=t^2, expand (t+h)^2-t^2=2th+h^2. Dividing by h gives 2t+h. Taking the limit gives x'(t)=2t. At t=3 the exact derivative is 6; the forward difference with h=0.1 is 6.1. The difference is a predictable approximation error, not a bug.

A function can be continuous without being differentiable. The absolute-value function |t| has left slope -1 and right slope +1 at zero, so it has no ordinary derivative there. A collision, clamp, or branch in a game model can create similarly nonsmooth behavior. Check the mathematical model before interpreting a numerical derivative.

Carry units through the derivative

Position measured in meters differentiated with respect to seconds yields meters/second. Differentiating velocity yields acceleration in meters/second^2. If an animation curve is parameterized by normalized progress u rather than time t, its derivative with respect to u is not automatically a physical velocity. The chain rule supplies dx/dt=(dx/du)*(du/dt).

Basic rules allow compositional reasoning

The derivative of t^n is n*t^(n-1) for the usual polynomial cases. Sums differentiate term by term. For a product fg, the derivative is f'g+fg'; it is not f'g'. For a composition f(g(t)), it is f'(g(t))*g'(t).

For x(t)=(2t+1)^3, the inner derivative is 2 and the outer derivative is 3(2t+1)^2, giving 6(2t+1)^2. Omitting the factor 2 treats the internal time scaling as though it had no effect. This same error appears in transformed geometry and parameterized animation.

A tangent approximation x(t+h) approximately equals x(t)+x'(t)h for small h. It is local. A large step through a sharply curved trajectory can make an accurate derivative produce a poor position prediction.

Numerical differences balance two errors

The centered estimate is (f(x+h)-f(x-h))/(2h). For sufficiently smooth functions, its leading truncation error is proportional to h^2, whereas the forward estimate commonly has error proportional to h. These orders describe a limiting regime with bounded higher derivatives, not every possible step size or function.

As h becomes tiny, subtraction of nearly equal floating-point numbers loses useful relative precision, and x+h may even round back to x. The reference rejects an unresolvable h. No single absolute perturbation is suitable for every coordinate scale. Use an error table over several h values, compare

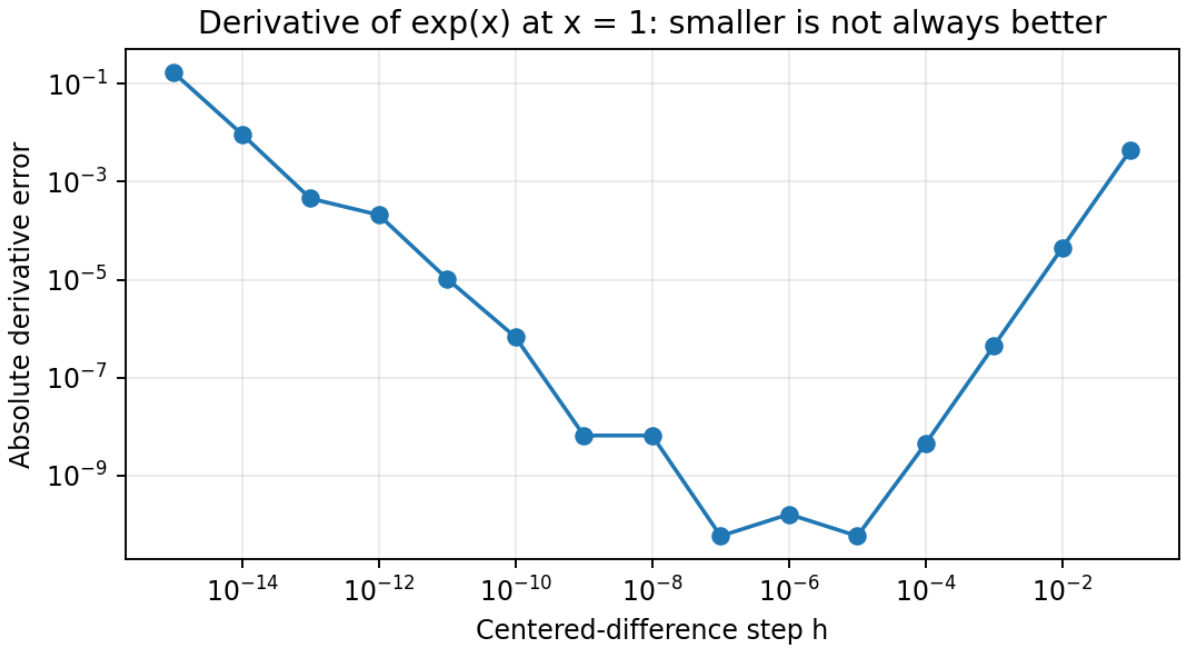
to an analytic derivative, and explain the observed region of improvement or deterioration. [R08, R04]

Worked derivation

Let f(t)=t^3 and evaluate at t=2. The exact derivative is 3t^2=12. Expanding the centered numerator gives (t+h)^3-(t-h)^3=6t^2*h+2h^3. After dividing by 2h, the estimate is 3t^2+h^2.

At h=0.1, 0.01 and 0.001, the exact-arithmetic estimates are 12.01, 12.0001 and 12.000001. Reducing h by ten reduces this truncation error by one hundred. Floating-point results follow that pattern only while rounding is small enough.

For x(t)=3t^2 meters with t in seconds, v(t)=6t m/s and a(t)=6 m/s^2. At t=2 s, the velocity is 12 m/s.



Centered differences for exp at x=1: reducing h eventually encounters floating-point limits. Unresolvable steps are excluded from the plot but recorded in the data.

Runnable demonstration

Run `python run_example.py 08.1` from the extracted package folder. Predict before executing.

```
from mathkit import centered_difference
f = lambda t: t ** 3
x = 2.0
exact = 3 * x * x
for h in (0.1, 0.01, 0.001):
    estimate = centered_difference(f, x, h)
    print(f"h={h:.3f} estimate={estimate:.6f} error={abs(estimate-exact):.6f}")
```

Recorded expected output:

```
h=0.100 estimate=12.010000 error=0.010000
h=0.010 estimate=12.000100 error=0.000100
h=0.001 estimate=12.000001 error=0.000001
```

Practical assignment

Original brief: Compare analytic and finite-difference derivatives of three functions over several step sizes. Pass: include units, an error table and an explanation of why the smallest tested h need not be best.

1. Implement centered_difference(f,x,h), requiring finite inputs, positive resolvable h, and finite function evaluations.
2. Compare analytic and numerical derivatives for t^2, t^3, and sin(t), using at least five step sizes. State radians for the sine argument.
3. Include the corner |t| at zero and explain why a centered value of zero does not establish differentiability.
4. Record absolute errors, units, and the approximation/rounding tradeoff. Do not smooth away a failed case without documenting it.

Coding slice: Work in `labs/student/lab_08_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

The smallest perturbation is not automatically the most accurate. A numerical slope at a corner is not proof that an ordinary derivative exists.

Independent transfer task

For x(t)=(3t-1)^2, derive velocity and acceleration, then evaluate at t=1.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Differentiate x(t)=3*t*t.
2. What units does acceleration have when position is in meters?
3. Why can excessively small finite-difference steps be inaccurate?
4. Differentiate (2t+1)^3.
5. Why is |t| not differentiable at t=0?

Assigned study

R08: derivative definition, basic rules, product and chain rules, and tangent approximation. R04: finite floating-point values and mathematical functions.

Original reading assignment retained: R08: differentiation, product rule and chain rule.

Lesson 08.2 — Integrals, accumulation and continuous probability

Learning outcome: Interpret integration as accumulated contribution.

Accumulation adds small contributions

An integral adds contributions across an interval. If v(t) is velocity, a tiny interval dt contributes approximately v(t)dt to displacement. The definite integral from a to b combines the limiting sum of such contributions. For a continuous integrand, this can be formalized as a limit of Riemann sums.

The integral is signed. A character moving forward and then backward can return to the starting position with zero net displacement while traveling a positive total distance. Integrating v gives displacement; integrating |v| gives distance in one-dimensional motion. The two are different objectives.

Derivatives and integrals are connected

If F is an antiderivative of f on the interval under the relevant regularity assumptions, the fundamental theorem of calculus gives the definite integral as F(b)-F(a). For f(t)=2t, use F(t)=t^2. The integral from 0 to 3 is 9, not the final velocity 6 multiplied by three seconds.

An indefinite integral represents a family of antiderivatives differing by a constant. Initial conditions fix that constant in a motion model. Integrating acceleration a(t) gives a velocity change; the initial velocity must still be added. Integrating velocity gives a position change; the initial position remains necessary.

Quadrature is a specified discrete approximation

Divide [a,b] into n equal panels of width h=(b-a)/n. The left rectangle rule samples f(a+i*h) for i=0,...,n-1 and multiplies their sum by h. The composite trapezoid rule gives half weight to the endpoints and full weight to internal samples.

For increasing f, left rectangles underestimate on a forward interval. For a curved function, trapezoids may overestimate or underestimate depending on curvature. Trapezoids integrate linear functions exactly in exact arithmetic; they do not integrate every smooth function exactly. Increasing n is useful only while model errors and numerical rounding are understood.

The reference allows a>b and returns the signed result using a negative panel width. Zero-width intervals return zero. It rejects nonpositive panel counts and unresolvable widths. Those are interface decisions that tests must match rather than infer from a library name.

Continuous probability uses areas, not point masses

A probability density p(x) is nonnegative and integrates to one over its support. Probability over an interval is the integral of the density there. For a continuous distribution, the probability at an exact single point can be zero even where the density is positive. Density values may exceed one because they are not themselves interval probabilities.

For a uniform position in [0,0.5] meters, the density is 2 per meter. The probability of [0.1,0.2] meters is 2*0.1=0.2. Units cancel. This same distinction later matters when sampling directions for rendering: a density must be specified with respect to a measure.

Check refinement against an independent answer

Use functions with known antiderivatives before trusting a complex simulation. At fixed total interval, compare n, 2n, and 4n panels. Report error against the analytic integral, not merely agreement between two versions of the same routine. A discontinuity or singularity requires a deliberate policy; blindly adding samples is not an explanation. [R08, R09]

Worked derivation

For f(t)=t^2 on [0,1], the exact integral is 1/3. With four panels, h=1/4. The left sum is h*(0^2+(1/4)^2+(1/2)^2+(3/4)^2)=0.21875.

The trapezoid sum is h*(0/2+(1/4)^2+(1/2)^2+(3/4)^2+1/2)=0.34375. Its error is 0.0104167. With eight panels, the result is 0.3359375 and the error is 0.00260417, one quarter as large. This is consistent with second-order composite trapezoid behavior for this smooth function.

For v(t)=2t on [0,3], the trapezoid rule is exact in exact arithmetic for any positive panel count, yielding displacement 9.

Runnable demonstration

Run `python run_example.py 08.2` from the extracted package folder. Predict before executing.

```
from mathkit import integrate
for n in (4, 8, 16):
    left = integrate(lambda t: t*t, 0, 1, n, "left")
    trap = integrate(lambda t: t*t, 0, 1, n, "trapezoid")
    print(f"n={n:2} left={left:.6f} trapezoid={trap:.6f}")
print("exact:", round(1/3, 6))
print("linear trapezoid:", integrate(lambda t: 2*t, 0, 3, 4))
```

Recorded expected output:

```
n= 4 left=0.218750 trapezoid=0.343750
n= 8 left=0.273438 trapezoid=0.335938
n=16 left=0.302734 trapezoid=0.333984
exact: 0.333333
linear trapezoid: 9.0
```

Practical assignment

Original brief: Implement rectangle and trapezoid integration for v(t)=2*t and a curved function. Pass: compare to an analytic integral, halve the step and report how error changes.

1. Implement integrate(f,a,b,n,method), supporting left and trapezoid rules with the documented signed-interval contract.
2. Compare t^2, 2t, and a chosen curved function against antiderivatives using n,2n,4n panels.
3. Include reversed limits, an equal-endpoint interval, invalid n, and a function returning a nonfinite value.

4. Write a displacement-versus-distance example and a normalized density calculation with units.

Coding slice: Work in `labs/student/lab_08_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A density is not a probability at a point. Net displacement is not total distance, and multiplying final velocity by total time is not a general integration rule.

Independent transfer task

Velocity is +2 m/s for one second and -2 m/s for the next second. Find displacement and distance.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Integrate constant velocity 5 over two seconds.
2. What is the integral of 2*t from 0 to 3?
3. Why can an integral be zero despite nonzero motion?
4. Can a probability density equal 2 without violating probability bounds?
5. What is the integral of t^2 from 1 to 0?

Assigned study

R08: definite integrals, both fundamental-theorem connections, probability applications, and numerical integration. R09: continuous distributions when available after the probability module begins.

Original reading assignment retained: R08: integration and the fundamental theorem; R23: Monte Carlo Integration introduction.

Lesson 08.3 — Newtonian mechanics and differential equations

Learning outcome: Model motion from forces with clear simplifying assumptions.

Specify the physical state and assumptions

For a one-dimensional particle, use state (x,v), where x is position and v is velocity. The differential equations are dx/dt=v and dv/dt=F_net/m for constant positive mass m. Initial position and velocity select a trajectory; forces alone do not tell you where the particle starts.

A model is useful because it omits details deliberately. This bundle's projectile model ignores drag, collision, rotation, and variation in gravity. The spring model is one-dimensional, linear, and attached to a fixed rest position. These are appropriate learning models, not claims that every game object or physical mechanism obeys them exactly.

Force, acceleration, and impulse have different units

Force measured in newtons equals kg*m/s^2. Dividing by mass gives acceleration. Momentum is m*v, measured in kg*m/s. Impulse is the integral of force over time, measured in N*s, and changes momentum.

An instantaneous gameplay knockback may be modeled as an impulse, producing delta_v=J/m. Applying the same force for a duration produces an impulse only after accounting for that duration. Adding a force directly to velocity mixes incompatible units and makes behavior accidentally depend on the update frequency.

Constant acceleration supplies an analytic reference

When a is constant, integrate dv/dt=a to get v(t)=v0+a*t. Integrating that velocity gives x(t)=x0+v0*t+0.5*a*t^2. These formulas are exact within the model in real arithmetic. They are useful independent reference cases for a numerical integrator.

Use a signed coordinate convention. With y-up, gravity can be a=-g. A negative velocity indicates downward motion; it is not inherently invalid. The exercise restricts elapsed time to t>=0 because it models forward simulation, even though the algebraic formula can be evaluated at negative t.

Springs and damping depend on state

For displacement x from equilibrium, a linear spring exerts -k*x. The stiffness k has units N/m. Linear viscous damping exerts -c*v, where c has units N*s/m. Thus acceleration is (-k*x-c*v)/m. Positive k restores toward equilibrium; nonnegative c opposes velocity.

For an undamped spring, omega=sqrt(k/m) has units radians/second, and the continuous period is 2*pi/omega when k>0. With initial state x0 and v0, the analytic undamped solution is x(t)=x0*cos(omega*t)+(v0/omega)*sin(omega*t). Its velocity follows by differentiation. The k=0 case needs a separate constant-velocity model rather than division by zero.

Energy checks test the model as well as the implementation

Mechanical energy is E=0.5*m*v^2+0.5*k*x^2. Differentiating along the continuous spring equations gives dE/dt=-c*v^2. With c=0 the ideal model conserves energy; with c>=0 it does not gain energy from damping. A numerical update can violate that property when the timestep or method is unsuitable.

A visually smooth curve is therefore weak evidence. Record equations, units, initial conditions, omitted effects, and an analytic comparison. Distinguish a model limitation from an integration error or an implementation bug. [R08, R28; derivations in this lesson]

Worked derivation

Take x0=0 m, v0=4 m/s, a=-10 m/s^2, and t=0.5 s. The analytic position is 0+4*0.5+0.5*(-10)*0.5^2=0.75 m; velocity is 4-10*0.5=-1 m/s. The object is above its start but already moving downward.

For m=2 kg, k=50 N/m, c=1 N*s/m, x=0.2 m, and v=0, the force is -10 N and acceleration -5 m/s^2. A semi-implicit step of 0.01 s gives v_new=-0.05 m/s and x_new=0.1995 m. That is a numerical step, not the exact damped-spring solution.

Runnable demonstration

Run `python run_example.py 08.3` from the extracted package folder. Predict before executing.

```
from mathkit import constant_acceleration, spring_step
x, v = constant_acceleration(0, 4, -10, 0.5)
print(f"projectile: x={x:.3f} m, v={v:.3f} m/s")
x, v = spring_step(0.2, 0, 2, 50, 1, 0.01, "semi")
print(f"spring: x={x:.5f} m, v={v:.5f} m/s")
```

Recorded expected output:

```
projectile: x=0.750 m, v=-1.000 m/s
spring: x=0.19950 m, v=-0.05000 m/s
```

Practical assignment

Original brief: Implement a particle under constant acceleration and a damped spring. Pass: compare the constant-acceleration trajectory to its analytic solution and state what collision, rotation and air-resistance effects are omitted.

1. Implement constant_acceleration(x0,v0,a,t), returning position and velocity with finite values and t>=0.
2. Use the spring model to run both damped and undamped examples. Record m,k,c,initial state, units, and omitted effects.
3. Derive the constant-acceleration solution and the continuous spring energy-rate identity on paper.
4. Compare the numerical undamped spring against its analytic sine/cosine trajectory at identical timestamps.

Coding slice: Work in `labs/student/lab_08_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Force is not velocity, mass is not weight, and energy conservation belongs first to a declared continuous model. Numerical damping or growth must be measured rather than hidden.

Independent transfer task

A 2 kg body receives a 3 N*s impulse. What velocity change results? Compare with 3 N applied for 0.5 s.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What acceleration results from 10 N on a 2 kg mass?
2. How does impulse differ from force?
3. Why must initial conditions accompany a differential equation?
4. Write the state equations for a linear damped spring about x=0.
5. What is dE/dt for the ideal linear damped spring?

Assigned study

R08: antiderivatives and differential-equation applications. R28 is a later simulation API comparison, not a substitute for deriving these particle equations. No Box2D installation is required here.

Original reading assignment retained: R28: simulation, bodies, forces and impulses; R08: differential-equation applications.

Lesson 08.4 — Time integration, stability and error

Learning outcome: Choose and test a numerical update rather than assuming smaller code is correct.

Update order changes the method

Explicit Euler evaluates all derivatives at the old state: x_new=x+v*h and v_new=v+a(x,v)*h. Semi-implicit Euler first updates velocity, then uses v_new in the position update. This is a different numerical method, not a stylistic rearrangement of equivalent assignments.

For constant acceleration, the exact position increment is v*h+0.5*a*h^2. Explicit Euler omits the second term; semi-implicit Euler adds a*h^2. One undershoots and the other overshoots according to the sign of a. Both are first-order methods for position over a fixed total duration in this setting.

Accuracy and stability ask different questions

Accuracy concerns how close the computed solution is to the intended reference. Stability concerns whether errors or modes grow uncontrollably under the discretized update. A method can be stable but inaccurate at a coarse step; a small local error can still accumulate into unacceptable long-term drift.

Fixed timesteps simplify comparison and replay, but they do not guarantee stability or bit-identical results across systems. A timestep needs units, a total horizon, and a relationship to the system's fastest relevant motion.

Derive a concrete instability instead of guessing

For an undamped spring, acceleration=-omega^2*x. Explicit Euler gives x_new=x+h*v and v_new=v-h*omega^2*x. Substitute these into E=0.5*m*v^2+0.5*k*x^2, using k=m*omega^2. The cross terms cancel and E_new=(1+omega^2*h^2)*E.

For any h>0 and nonzero energy, the factor is greater than one. Repeated explicit steps therefore grow the oscillator's energy, even though the continuous model conserves it. At a fixed physical horizon, refining h reduces this drift; at a fixed nonzero h over arbitrarily long time, it persists.

For semi-implicit Euler, the oscillator update matrix has determinant 1 and trace 2-(omega*h)^2. For 0<omega*h<2, its eigenvalues are a distinct complex-conjugate pair of unit magnitude, yielding bounded motion for this particular linear system. At the boundary omega*h=2, boundedness is not generally guaranteed; beyond it there is a growing mode. Do not transfer this exact condition unchanged to nonlinear or damped systems.

Measure error at matching physical times

Comparing 100 steps at h=0.01 with 100 steps at h=0.005 compares one second with half a second. For a refinement study, keep T fixed and use N=T/h steps, choosing compatible h values. Include the initial state and compute timestamps as i*h rather than accumulating an unrelated wall-clock timer.

Record maximum position error, final position error, energy ratios, and nonfinite or divergence status. A deliberately unstable case belongs in the report. Do not delete it merely because it spoils the plot.

Separate the error sources

Model error comes from omitted or simplified physics. Discretization error comes from replacing the continuous evolution by a numerical update. Roundoff error comes from finite arithmetic. Measurement error belongs to observations used to fit or evaluate the model. More integration steps may reduce one source without changing the others.

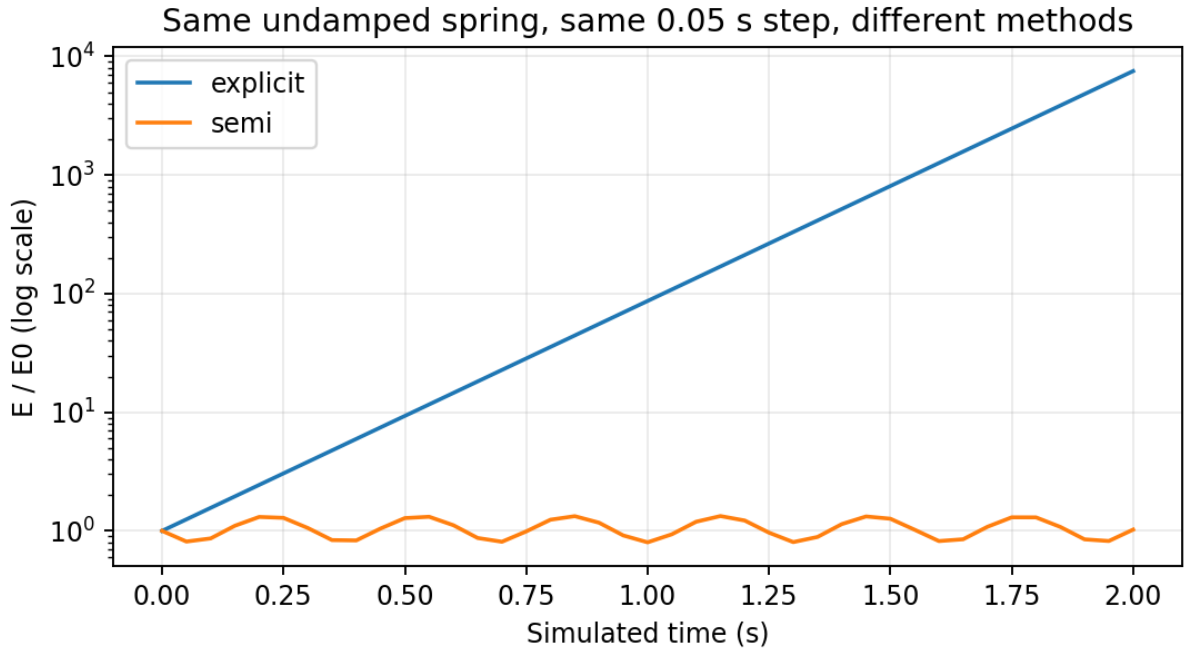
For larger projects, higher-order and implicit methods offer different cost/stability tradeoffs. This lesson does not claim that semi-implicit Euler is universally superior; it establishes a repeatable way to compare methods on a stated problem. [R08, R07; oscillator derivations in this lesson]

Worked derivation

Choose m=1, k=100, c=0, x0=1, v0=0. Then omega=10 and E0=50. With h=0.05, omega*h=0.5. Explicit Euler multiplies energy by 1.25 each step, so after 20 steps E/E0=1.25^20, approximately 86.736.

The semi-implicit run at the same h and one-second horizon has a bounded but oscillating energy; the supplied run ends near E/E0=0.800291. Ending below the initial energy does not prove exact energy conservation or monotonic damping.

For h=0.25, omega*h=2.5 exceeds the linear semi-implicit stability bound. Include this adverse case over a bounded horizon and report its growth rather than extrapolating indefinitely into overflow.



One chosen undamped oscillator, identical timestep and final time. Explicit Euler grows energy; semi-implicit Euler oscillates in energy and is not exact.

Runnable demonstration

Run `python run_example.py 08.4` from the extracted package folder. Predict before executing.

```
from mathkit import spring_step
for method in ("explicit", "semi"):
    x, v = 1.0, 0.0
    for _ in range(20):
        x, v = spring_step(x, v, 1, 100, 0, 0.05, method)
    energy = 0.5*v*v + 50*x*x
    print(f"{method}: E/E0={energy/50:.6f}")
print(f"explicit prediction: {(1 + 100*0.05**2)**20:.6f}")
```

Recorded expected output:

```
explicit: E/E0=86.736174
semi: E/E0=0.800291
explicit prediction: 86.736174
```

Practical assignment

Original brief: Compare explicit and semi-implicit Euler on an undamped spring at three timesteps. Pass: plot or tabulate position and energy error, identify an unstable case, and avoid calling visual smoothness proof of physical accuracy.

1. Implement spring_step(x,v,mass,stiffness,damping,dt,method), supporting explicit and semi updates and rejecting invalid physical parameters.
2. Compare both methods at h=0.1,0.05,0.025 over the same two-second horizon, against the analytic undamped solution.
3. Add an unstable semi case h=0.25 for m=1,k=100 and a bounded run length. Preserve energy and error tables, not only an animation.
4. Derive the explicit energy-growth identity; explain at least one model error that timestep refinement cannot repair.

Coding slice: Work in `labs/student/lab_08_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Equal step counts are not equal simulation durations. Smoothness is not accuracy, and a fixed step does not make an unstable update stable.

Independent transfer task

For m=1,k=25 and an undamped spring, what open interval of positive h satisfies the derived semi-implicit condition?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Which velocity does semi-implicit Euler use for the position update?
2. Does a fixed timestep guarantee a stable simulation?
3. What is a useful check beyond watching the animation?

4. For explicit Euler on an undamped spring, what is the one-step energy factor?
5. Why must a refinement study hold the final physical time fixed?

Assigned study

R08: differential equations and local approximations. R07: eigenvalues for the small update matrix. The original roadmap cross-references animation through R19/R23; use the explicit oscillator derivation here rather than assuming R23 contains a general simulation course.

Original reading assignment retained: R23: physically based animation topics via R19; R28: simulation stepping guidance.